FH Salzburg
MultiMediaTechnology

Concurrent & Distributed Systems, 2. Klausur am 22.01.2024

Nachname / Vorname: Kenny Sason

Hinweise: Schreiben Sie leserlich. Antworten Sie knapp und unter Verwendung von Fachterminologie zu begründen. Internetfähige Geräte und schriftliche Unterlagen sind nicht erlaubt (closed books).

Zeitvorgabe: max. Bearbeitungszeit 45 Minuten

Gesamtpunkte: 18

8/18

1. Welche Aufgabe haben die folgenden Komponenten eines Rechners? (0,5P pro Antwort)
   a. Befehlszähler
   b. Steuerwerk
   c. ALU
   d. Allgemeine Register

1,5

a, zeigt auf aktuellen Befehl im Speicher
b., Löst Befehle für das Rechenwerk auf um Schaltinstruktionen zu erstellen (steuert Befehlsabfolge
c, führt Rechenoperationen durch (Rechenwerk) + logische Operationen
d, für das Zwischenspeichern verantwortlich was wird zwischengespeichert

2. Moderne CPUs haben L1-/L2- und L3 Caches?
   a. Warum gibt es so viele verschieden Caches? (0,5P)
   b. Was versteht man unter dem Begriff Lokalität im Zusammenhang mit Caches. (0,5P)
   c. Angenommen Sie greifen lesend auf eine int-Variable im Hauptspeicher zu. Wie und wo kommen die L1-/L2- und L3 Caches zum Zuge. (1P)

1

a, wir haben mehrere Caches da in L1 z.B. ist die schnellste Zugriffszeit hat aber am wenigsten Speicherkapazität und L3 langsamste Zugriffszeit (von den 3) aber am meisten Speicherkapazität, um zu kategorisieren welche Daten extrem oft gebraucht -> L1 und oft aber nicht ganz so oft -> L3

b., Darunter versteht man wieso Caches Funktionieren, wir haben die zeitliche Lokalität = Operationen die zuletzt verwendet wurden, werden wahrscheinlich bald wieder verwendet, und räumliche Lokalität = Operationen die hintereinander ausgeführt werden liegen auch oft hintereinander im Cache. 2

+C

3. Interpretieren Sie die folgende Formel. (2P)

$$T_p(n) \geq f * T'(n) + \frac{(1-f) * T'(n)}{p}$$

Amdahls Law

1

P = Anzahl Prozessoren

f = sequenzieller Anteil (kann nicht parallelisiert werden) $0 \leq f < 1$

$T_p(n)$ = Problemgröße

$T'(n)$ = Speed up

was ist $(1-f)$?

Der Speed up sinkt stark, auch nur bei einem geringen Anteil von f.
Haben wir Doppelt so viele Prozessoren, kann die Aufgabe höchstens dannell so schnell erledigt werden

Problem bei Amdahls Law ist = Cachen werden nicht berücksichtigt + es wird nicht berücksichtigt, dass sich Problemgrößen ändern können

4. Beantworten sie die folgenden Fragen anhand dieses Codebeispiels.

```
public void Method1()
{
    // Line 1
    int i=4;

    // Line 2
    int y=2;

    //Line 3
    class1 cls1 = new class1();
}
```

a. Wo liegt die Variable y im Speicher? (0,5P)
b. Was ist cls1 und wo liegt diese im Speicher? (0,5P)
c. Kann ein data-race beim Zugriff auf cls1 mit mehreren Threads entstehen? Begründung? (1P)

a, im Stack

b, ist ein Object und liegt in Heap cls1 liegt am Stack

c, Ja, das kann durchaus entstehen, da die Function nicht gelockt ist. Mehre Aktivitäten können auf die selbe Klasse zugreifen wollen und eine oder mehr kann sie auch verändern wollen

1

welche Function?

5. Frage zu Threads. Beantworten Sie die Fragen mit Code-Snippets in C#
   a. Erstellen eines Threads der die Methode work() ohne Parameter aufruft. (0,5P)
   b. Erstellen eines Threads der die Methode work(int 1, int 2) aufruft. (0,5P)
   c. Code-Snippet das auf die Beendigung von Thread t wartet. (0,5P)
   d. Erstellen einer Semaphore die mit 1 initialisiert ist und zur Schutz von kritischen Bereichen dient. (0,5P)

a, Thread t1 = Thread(() => work());

b, Thread t1 = Thread(() => work(1, 2));

c, t.Join();

d, Semaphore sem = new Semaphore(1, 1);

2

6. Erläutern Sie die Verwendung des **Monitors** in folgender Implementierung Producer/Consumer Problems. (2P)

```
using System;
using System.Threading;

class Program {
    static int[] buffer = new int[10];
    static int count = 0;
    static int inIndex = 0;
    static int outIndex = 0;

    static void Main(string[] args) {
        Thread producer = new Thread(Produce);
        Thread consumer = new Thread(Consume);

        producer.Start();
        consumer.Start();

        producer.Join();
        consumer.Join();
    }

    static void Produce() {
        for (int i = 0; i < 100; i++) {
            lock (buffer) {
                while (count == buffer.Length) {
                    Monitor.Wait(buffer);
                }

                buffer[inIndex] = i;
                inIndex = (inIndex + 1) % buffer.Length;
                count++;

                Monitor.Pulse(buffer);
            }
        }
    }

    static void Consume() {
        for (int i = 0; i < 100; i++) {
            lock (buffer) {
                while (count == 0) {
                    Monitor.Wait(buffer);
                }

                int value = buffer[outIndex];
                outIndex = (outIndex + 1) % buffer.Length;
                count--;

                Console.WriteLine("Consumed: " + value);

                Monitor.Pulse(buffer);
            }
        }
    }
}
```

1

CS

CS

+ Pulse / Wait

Der Monitor wird verwendet, damit nicht mehr als 1 Thread auf ~~aus~~ eine Funktion zugreifen kann. In diesem Fall verhindert der Monitor, ~~dass~~ dass versucht wird, Daten aus einen leeren Buffer zu lesen und Daten in einen vollen Buffer zu speichern

lock()
{ = eine Critical Section, ~~hier~~ hier kann nur 1 Thread zu gleicher Zeit drauf zugreifen
}

7. Schreiben Sie eine Task, die die nachfolgende Methode asynchron aufruft und das Ergebnis auf der Konsole ausgibt. (2P)

```
public async Task<int> AddAsync(int a, int b) {
    await Task.Delay(1000); // simulate a long-running operation
    return a + b;
}
```

~~public await~~

8. Schreiben Sie zu diesem Server einen passenden Client. (2P)

```
using System;
using System.Net;
using System.Net.Sockets;

class UDPServer {
    static void Main(string[] args) {
        Socket server = new Socket(AddressFamily.InterNetwork, SocketType.Dgram,
                                   ProtocolType.Udp);
        IPEndPoint localEP = new IPEndPoint(IPAddress.Any, 1234);

        server.Bind(localEP);

        byte[] buffer = new byte[1024];

        while (true) {
            EndPoint remoteEP = new IPEndPoint(IPAddress.Any, 0);

            int bytesReceived = server.ReceiveFrom(buffer, ref remoteEP);
            string message = Encoding.ASCII.GetString(buffer, 0, bytesReceived);
            Console.WriteLine("Received message: " + message);

            byte[] response = Encoding.ASCII.GetBytes("Message received");
            server.SendTo(response, remoteEP);
        }
    }
}
```

0,5

```
class UDPClient {
    static void Main (string[] args {
        Socket client = new Socket( AdressFamily.InterNetwork, SocketType.Dgram,
                                    ProtocolType.Udp);

        IPEndPoint localEP = new IPEndPoint ( IPAddress.Any);
        client.Bind (localEP);
        byte[] buffer = new byte [1024];

        while (true) {
            EndPoint remoteEP = new IPEndPoint ( IPAddress.Any 0);
            int bytesSent = client.SendTO(buffer, ref remoteEP);
            string message = Encoding.ASCII.GetString (buffer, 0, bytesSent);
            Console.WriteLine("Sent message: " + message);
            byte[] response = Encoding.ASCII.GetBytes ("message sent");
            client.ReceiveFrom (response, ref remoteEP);
```

9. Ihre Aufgabe ist es, den folgenden gRPC Dienst zu erweitern. Neu hinzukommen soll eine Methode SearchBookTitle() zum Suchen eines Buches nach dem Titel.

   a. Erweitern Sie die IDL um SearchBookTitle(). (1P)

```
syntax = "proto3";

service Library {
  rpc GetBookInfo (BookInfoRequest)
    returns (BookInfoResponse) {}
}

message BookInfoRequest {
  string isbn = 1;
}

message BookInfoResponse {
  string title = 1;
  string author = 2;
  double price = 3;
}
```

~~message SearchBookTitle {~~
~~string title = 1;~~
}

message SearchBookTitle (string title)
{
var search;
~~if (title~~

   b. Ergänzen Sie den folgenden Code, um den Aufruf der Methode SearchBookTitle() aufruft. Die entsprechenden Stellen sind „fett“ markiert. (1P)

```
using System;
using Grpc.Core;


class Program
{
    static void Main(string[] args)
    {
            Channel channel = new Channel("127.0.0.1:30052", …);
            var client = new Library.LibraryClient(channel);

            Console.WriteLine("Enter title ");
            string title = Console.ReadLine();

            // erstellen des requests
var info = SearchBookTitle (title);


            // gRPC anfrage stellen
rpc GetBookInfo(info)


            // response ausgeben
returns(info)


            channel.ShutdownAsync().Wait();
        }
    }
```